Lecture 13

21 October 2026 23:06

Now we are at the **final two steps** of the video to turn this into a full, production-style GPT!

Step 1: Dropout (Preventing Overfitting) 🛡️

Right now, as we make our model deeper and smarter, it runs the risk of **overfitting** (memorizing the Shakespeare text word-for-word instead of learning the underlying grammar and style).
(we have seen this before, how our model should be predicting by learning and not by memorizing)

How this is take palce with example :

Dropout fixes this: V·IMP

V IMP:

1. At every training step, it **randomly shuts off 20% of the neurons** (sets them to 0).

- **Analogy:** Imagine a sports team where random players are benched during practice. The remaining players are forced to step up and learn to play without relying on any single star player. It makes the entire network robust!

In PyTorch, Karpathy adds nn.Dropout(dropout) (with dropout = 0.2) in **3 specific places**:

1. **Inside Head (on attention weights):**

Python

```
self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))
self.dropout = nn.Dropout(dropout)     ----> NEW
...
wei = F.softmax(wei, dim=-1)   ---> NEW
wei = self.dropout(wei)  # randomly prevent some tokens from communicating
```

2. **Inside MultiHeadAttention (on projected heads):**

Python

```
self.proj = nn.Linear(n_embd, n_embd)
self.dropout = nn.Dropout(dropout) ----> NEW
...
out = self.dropout(self.proj(out))   ----> NEW
```

3. **Inside FeedForward (at the end of thinking):**

```
python
self.net = nn.Sequential(
    nn.Linear(n_embd, 4 * n_embd),
    nn.ReLU(),
    nn.Linear(4 * n_embd, n_embd),
    nn.Dropout(dropout),  # <-- add dropout here!
)
```

And the hyperparameters have been changed to this:

```
# Scaled-up Hyperparameters:
batch_size = 64     # 64 parallel sequences (was 32)
block_size = 256    # 256 characters of context (was 8!)
max_iters = 5000
eval_interval = 500
learning_rate = 3e-4 # standard transformer learning rate
eval_iters = 200
n_embd = 384       # 384 features per token (was 32!)
n_head = 6         # 6 attention heads (384 // 6 = 64 head_size)
n_layer = 6        # 6 transformer blocks deep (was 3)
dropout = 0.2      # 20% dropout
```

And we are good too go

The model will take 6 to 8 min to train and will return a text, but the text seems good but not-sensical, as the training loss is down to **~2.5** to **~1.48** roughly,

As we have made only the decoder block and not the encoder + decoder block and also in the decoder transformer we lack cross-attention which will help all the token to talk to each other and not to see the next token like we did in the part before the taking of the query and the key.